

CONTENTS

Representación Interna de Datos	
Guía detallada desde cero — con los detalles que salvan el parcial	
0. Por qué existe este problema	
1. Enteros sin signo	
2. El problema de representar negativos	
3. Valor Absoluto y Signo (VA+S)	
4. Complemento a 1 (C1)	
5. Complemento a 2 (C2) — el que usa el hardware real	
5.1 Por qué la MISMA tira de bits vale distinto en cada formato	
6. Desplazamiento (exceso-d)	
7. Tabla resumen — rangos y ceros	
8. Flags de la suma en C2: Z, N, C, V	
9. Punto fijo	
10. Por qué hace falta punto flotante	
11. IEEE 754 — estructura	
11.1 Codificar — ejemplo completo	
11.2 Decodificar — ejemplo	
12. Casos especiales de IEEE 754 (memorizar)	
13. Suma en IEEE 754 — procedimiento completo	

13.1 El “número absorbido” — la trampa más recurrente en examen

14. Error de redondeo

15. Ejercicios de repaso rápido (con respuesta)

16. Resumen ultra-rápido (2 minutos)

Representación Interna de Datos

GUÍA DETALLADA DESDE CERO — CON LOS DETALLES QUE SALVAN EL PARCIAL

Arquitectura de Computadoras — FING UdelaR

Enteros sin signo · VA+S · Complemento a 1 · Complemento a 2 · Desplazamiento · Flags
Z/N/C/V · Punto fijo · IEEE 754 (simple y media precisión)

Cómo usar esta guía: cada tema tiene una definición corta, el procedimiento paso a paso, al menos dos ejemplos resueltos, y los detalles que suelen explicarse mal o rápido en clase pero que definen si un ejercicio de examen sale bien o mal. Los recuadros amarillos son ejemplos, los verdes son trucos de verificación, y los rojos son errores frecuentes de tolerancia cero.

0. Por qué existe este problema

Una computadora solo tiene bits: 0 y 1. Cualquier dato — un entero, un número negativo, un número con decimales, un carácter — tiene que reducirse a una tira fija de n bits. El problema central de este tema es: **dado un número real que querés guardar, ¿qué tira de bits le corresponde, y cómo la volvés a leer sin ambigüedad?**

Cada formato que vas a ver en esta guía (VA+S, C1, C2, Desplazamiento, punto fijo, IEEE 754) es una convención distinta para resolver exactamente ese mismo problema. No hay una única forma “correcta” — cada una tiene un trade-off distinto entre simplicidad de hardware, rango representable, y facilidad de comparación.

Idea que hay que tener siempre presente: una misma tira de bits puede significar números completamente distintos según qué formato se esté usando para leerla. El patrón de bits, por sí solo, no tiene significado.

1. Enteros sin signo

Definición: los n bits disponibles son todos magnitud — no hay ningún bit reservado para el signo.

Fórmula del valor: cada bit vale su posición en potencias de 2. El valor es la suma de las potencias de 2 donde hay un bit en 1.

Rango con n bits: de 0 a $2^n - 1$.

Ejemplo 1 — decodificar 00000101 (8 bits): $4 + 1 = 5$. Rango en 8 bits: 0 a 255.

Ejemplo 2 — codificar 200 en 8 bits: $200 = 128 + 64 + 8 = 2^7 + 2^6 + 2^3 \rightarrow 11001000$.

Verificación: $128 + 64 + 8 = 200 \checkmark$

Error frecuente: antes de representar un número, hay que chequear que entre en el rango. Si te piden representar 300 en 8 bits sin signo (máximo 255), la respuesta correcta es “no representable” — nunca truncar el número y seguir como si nada.

2. El problema de representar negativos

Con enteros sin signo no hay forma de indicar “menos”. Existen 4 convenciones estándar, todas usando **n bits totales**, donde el primer bit (más a la izquierda) se llama **MSB** (bit más significativo):

- **Valor Absoluto y Signo (VA+S):** el signo es una etiqueta separada de la magnitud.
- **Complemento a 1 (C1):** el negativo se construye invirtiendo todos los bits del positivo.
- **Complemento a 2 (C2):** el negativo es C1 más 1. Es el que usa el hardware real.
- **Desplazamiento (exceso-d):** se le suma un sesgo fijo al número y se guarda como sin signo.

3. Valor Absoluto y Signo (VA+S)

Definición: el MSB es una etiqueta de signo pura (0=positivo, 1=negativo), completamente separada del resto. Los $n-1$ bits restantes son la magnitud, sin ninguna transformación.

Codificar: (1) escribir la magnitud en binario con $n-1$ bits, (2) anteponer el bit de signo.

Ejemplo — representar -5 en 8 bits: magnitud de 5 = 0000101 (7 bits) $\rightarrow$ antepongo signo=1 $\rightarrow 10000101$

Decodificar: (1) mirar el MSB, (2) leer el resto como sin signo (magnitud), (3) aplicar el signo.

Ejemplo — decodificar 10000101 (8 bits): signo=1 (negativo), magnitud=0000101=5 → resultado = **-5**

Rango: $-(2^{n-1}-1)$ a $+(2^{n-1}-1)$. En 8 bits: -127 a +127. Simétrico.

Detalle clave — doble cero: 00000000 (+0) y 10000000 (-0) son dos tiras de bits distintas que representan el mismo valor matemático. Esa combinación “desperdiciada” es la razón por la que el rango es simétrico en vez de aprovechar un valor negativo extra.

4. Complemento a 1 (C1)

Definición: si el número es positivo, igual que VA+S. Si es negativo, se toma la versión positiva y se **invierten todos los bits** (los n bits completos, MSB incluido).

Codificar -5 en 8 bits: +5=00000101 → invierto → **11111010**

Decodificar 11111010: MSB=1 (negativo) → invierto todo → 00000101=5 → resultado=**-5**

Rango: igual que VA+S: -127 a +127 (8 bits).

También tiene doble cero: 00000000=+0, y 11111111 (invertido da 0) = -0.

Lectura directa con fórmula (sin invertir bit a bit): en C1, el bit de signo pesa $-(2^{n-1}-1)$; el resto pesa positivo normal.

Verificación — 11111010 (8 bits): peso del signo= $-(2^7-1)=-127$; resto= $1111010_2=122$; valor= $-127+122=-5$ ✓ coincide.

Truco para no memorizar el peso: la tira de todos unos (11111111) invertida da 0 → representa -0. Ese “-0” es la firma de que el rango negativo es “uno más corto” que en C2.

5. Complemento a 2 (C2) — el que usa el hardware real

Definición: positivo igual que los anteriores. Negativo: C1 (invertir todos los bits) **más 1**.

Codificar -5 en 8 bits: +5=00000101 → invierto=11111010 → sumo 1 → **11111011**

Decodificar 11111011: MSB=1 → invierto=00000100 → sumo 1 → 00000101=5 → resultado=**-5**

Rango: -2^{n-1} a $+2^{n-1}-1$ (asimétrico). En 8 bits: -128 a $+127$. Un negativo más que positivos.

Un solo cero: $00000000=0$, sin otra tira que también valga 0. Esa combinación que en VA+S/C1 se “perdía” en el -0 , acá se reutiliza para el -128 extra.

Lectura directa con fórmula: el bit de signo pesa -2^{n-1} ; el resto pesa positivo normal.

Verificación — 11111011 : peso del signo $= -2^7 = -128$; resto $= 1111011_2 = 123$;
valor $= -128 + 123 = -5$ ✓ (mismo resultado, sin invertir nada)

Por qué el hardware usa C2: con VA+S y C1, sumar requiere detectar signos por separado y aplicar lógica de resta. Con C2, el sumador binario normal (el mismo de sin signo) da directamente el resultado correcto — el acarreo que “cae” del MSB simplemente se descarta.

5.1 POR QUÉ LA MISMA TIRA DE BITS VALE DISTINTO EN CADA FORMATO

Tira de 4 bits **1011**:

Formato	Procedimiento	Resultado
VA+S	signo=1, magnitud=011=3	-3
C1	invierdo todo: $1011 \rightarrow 0100 = 4$	-4
C2	invierdo (0100) y sumo 1 = $0101 = 5$	-5

Con la fórmula directa (4 bits): VA+S no suma el signo (etiqueta aparte). C1: signo pesa $-(2^3-1) = -7 \rightarrow -7 + 011 (=3) = -4$ ✓. C2: signo pesa $-2^3 = -8 \rightarrow -8 + 011 (=3) = -5$ ✓.

6. Desplazamiento (exceso-d)

Definición: se le suma un valor fijo d (el sesgo) al número real N, y $N+d$ se codifica como sin signo normal. El convenio estándar usa $d=2^{n-1}$.

Codificar -5 en 8 bits (d=128): $-5 + 128 = 123 \rightarrow$ binario sin signo de 123 $\rightarrow$ **01111011**

Decodificar 01111011 (d=128): sin signo=123 $\rightarrow 123 - 128 = -5$

Rango: -2^{n-1} a $+2^{n-1}-1$ (igual forma que C2). En 8 bits: -128 a $+127$.

Truco de verificación cruzada (muy usado en exámenes): con $d=2^{n-1}$, **Desplazamiento(N)** = **Complemento a 2(N)** con el bit de signo invertido. Si tus dos resultados no cumplen esto, hay un error en alguno.

Chequeo: C2(-5) en 8 bits=11111011. Invierto el MSB → 01111011. Coincide con Desplazamiento(-5) ✓

Por qué existe: el patrón de bits queda en orden monótono creciente con el valor real, permitiendo comparar como enteros sin signo, sin decodificar el signo aparte — exactamente por qué IEEE 754 usa Desplazamiento para el exponente (Parte 11).

7. Tabla resumen — rangos y ceros

Formato	Rango (n bits)	Rango en 8 bits	Cantidad de ceros
Sin signo	0 a 2^n-1	0 a 255	uno
VA+S	$-(2^{n-1}-1)$ a $+(2^{n-1}-1)$	-127 a +127	dos (+0 y -0)
C1	$-(2^{n-1}-1)$ a $+(2^{n-1}-1)$	-127 a +127	dos (+0 y -0)
C2	-2^{n-1} a $+2^{n-1}-1$	-128 a +127	uno
Desplazamiento ($d=2^{n-1}$)	-2^{n-1} a $+2^{n-1}-1$	-128 a +127	uno

Pregunta clásica de examen: “¿qué formatos tienen distinta cantidad de positivos que negativos?” — La trampa es que VA+S y C1 NO tienen ese desbalance en sí (tienen dos ceros, uno de cada signo). El que sí tiene desbalance real es C2 (y Desplazamiento estándar): al haber un solo cero, sobra espacio para un negativo extra.

8. Flags de la suma en C2: Z, N, C, V

Flag	Nombre	Se activa cuando...
Z	Zero	el resultado (truncado al ancho de bits) es exactamente 0
N	Negative	el MSB del resultado es 1
C	Carry	hubo acarreo más allá del bit más significativo (desborde sin signo)
V	oVerflow	hubo desborde con signo: ver regla abajo

La regla de V — la que más se confunde: $V=1$ **si y solo si** ambos operandos tienen el **mismo signo** y el resultado tiene un **signo distinto** al de ellos. Si los operandos tienen signos distintos entre sí, nunca hay overflow — el resultado matemáticamente siempre cae entre los dos operandos.

Con overflow (8 bits, C2): $0x50 (+80) + 0x50 (+80) = 0xA0$. En C2 de 8 bits, $0xA0$ tiene $MSB=1 \rightarrow$ se lee como -96 . Ambos operandos positivos, resultado negativo $\rightarrow V=1$. Tiene sentido: $80+80=160$, fuera del rango (-128 a 127).

Sin overflow: $0x50 (+80) + 0xF0 (-16)$. Signos distintos $\rightarrow$ automáticamente $V=0$, sin calcular nada más.

Fórmula alternativa (por bits): $V = (\text{acarreo que entra al bit de signo}) \text{ XOR } (\text{acarreo que sale del bit de signo})$.

C y V son independientes: podés tener $C=1$ sin V (dos negativos grandes que generan acarreo pero el resultado sigue siendo negativo válido), o $V=1$ sin C (dos positivos que se pasan de rango sin acarreo, como $80+80$).

En una **resta** ($A-B$, implementada como $A + \text{complemento a 2 de } B$), C cambia de interpretación: $C=1$ significa “no hubo préstamo (borrow)”, es decir $A \geq B$ en aritmética sin signo. No leerlo como “hubo error” automáticamente.

9. Punto fijo

Definición: n bits totales = 1 signo + m parte entera + k parte fraccional (todo en C2). El punto binario está en una posición fija, no se guarda explícitamente.

Procedimiento: (1) multiplicar el número real por 2^k (corre el punto k lugares, da un entero), (2) convertir ese entero a C2 normal con los n bits, (3) ese patrón es la representación final.

Representar 5.125 en 16 bits, 6 fraccionales: $5.125 \times 2^6 = 328$. 328 en binario (16 bits) = 0000000101001000. **Resultado: 0000 0001 0100 1000**

Representar -8.75 en el mismo formato: $-8.75 \times 64 = -560$. 560 en binario = 00000001000110000 → C2: invierto (1111110111001111) y sumo 1 → **1111 1101 1101 0000**

Verificación: 0xFDD0 sin signo = 64976, $65536 - 64976 = 560$, $560/64 = 8.75$ ✓

Rango y precisión: máximo = $2^m - 1$ (entera) + $(2^k - 1)/2^k$ (fraccional). La precisión (salto mínimo) es siempre exactamente $1/2^k$, constante en todo el rango — a diferencia de punto flotante (Parte 14).

10. Por qué hace falta punto flotante

Los enteros (incluso punto fijo) no pueden cubrir a la vez números muy grandes y muy chicos con buena precisión, porque la cantidad de bits fraccionales es fija. Punto flotante resuelve esto con notación científica en binario: signo + exponente (cuánto “correr” el punto) + mantisa (dígitos significativos).

11. IEEE 754 — estructura

Definición: cualquier número ($\neq 0$) se normaliza a $1.\text{mantisa} \times 2^{\text{exponente}}$. El “1.” está implícito y NO se guarda (en binario normalizado el primer dígito siempre es 1 — un bit extra de precisión gratis).

[1 bit signo | exponente (con Desplazamiento) | mantisa]

Formato	Signo	Exponente	Mantisa	Sesgo (d)
Media precisión (half)	1 bit	5 bits	10 bits	15
Simple precisión (float)	1 bit	8 bits	23 bits	127

El exponente se guarda con Desplazamiento (Parte 6) — mismo mecanismo, mismo propósito: comparar dos floats normalizados del mismo signo como enteros sin signo.

11.1 CODIFICAR — EJEMPLO COMPLETO

Representar 0.5 en simple precisión: normalizo $0.5 = 1.0 \times 2^{-1}$. Exp real=-1, sesgado=-1+127=126=01111110. Mantisa=000...0 (23 ceros). Signo=0.

Resultado: 0 01111110 00000000000000000000000 = 0x3F000000

Representar -6.25 en simple precisión: $6.25 = 110.01_2 = 1.1001 \times 2^2$. Exp real=2, sesgado=129=10000001. Mantisa=1001+19 ceros. Signo=1.

Resultado: 1 10000001 10010000000000000000000 = 0xC0C80000

11.2 DECODIFICAR — EJEMPLO

Decodificar 0x3F000000: bits 0 01111110 00000000000000000000000. Signo=0. Exp sesgado=126 → real=-1. Mantisa=0 → significando=1.0. Valor= $1.0 \times 2^{-1} = 0.5$ ✓

12. Casos especiales de IEEE 754 (memorizar)

Exponente guardado	Mantisa	Significa	Valor
todo 0	todo 0	Cero (con signo)	+0 o -0
todo 0	≠0	Desnormalizado	$0.\text{mantisa} \times 2^{(1-\text{sesgo})}$
todo 1	todo 0	Infinito (con signo)	$\pm\infty$
todo 1	≠0	NaN	resultado inválido

Por qué existen los desnormalizados — underflow gradual: el menor normalizado positivo en simple precisión es 1.0×2^{-126} . Sin desnormalizados, cualquier número más chico se redondearía abruptamente a 0. Los desnormalizados usan el mismo exponente fijo $2^{(1-\text{sesgo})}$ y dejan que la mantisa (sin el “1.” implícito) represente valores cada vez más cercanos a 0.

Decodificar 0x00000001 (simple precisión): signo=0, exponente=todo 0 → desnormalizado. Mantisa=...001 (último bit en 1). Valor= $2^{-23} \times 2^{-126} = 2^{-149}$ — el menor número positivo representable en todo el formato.

13. Suma en IEEE 754 — procedimiento completo

1. Descomponer ambos números en signo, exponente real, y significando (1.mantisa).
2. Igualar exponentes: el de exponente más chico corre su mantisa a la derecha tantas posiciones como la diferencia.
3. Sumar los significandos ya alineados (o restar si signos distintos).
4. Normalizar el resultado.
5. Re-empaquetar: signo, exponente sesgado, mantisa truncada.

Sumar 3EE00000h + 3D800000h (simple precisión):

3EE00000h = 0 01111101 11000...0 → exp real=-2, significando= 1.11×2^{-2}

3D800000h = 0 01111011 000...0 → exp real=-4, significando= 1.0×2^{-4}

Igualo al mayor exponente (-2): $1.0 \times 2^{-4} = 0.01 \times 2^{-2}$

Sumo: $1.11 \times 2^{-2} + 0.01 \times 2^{-2} = 10.00 \times 2^{-2}$

Normalizo: $10.00 \times 2^{-2} = 1.000 \times 2^{-1}$

Re-empaqueteo: signo=0, exp=126=01111110, mantisa=0...0 → **Resultado: 3F000000h**

Verificación: 3EE00000h=0.4375; 3D800000h=0.0625; suma=0.5 ✓ (0x3F000000 es la constante estándar de 0.5f)

13.1 EL “NÚMERO ABSORBIDO” — LA TRAMPA MÁS RECURRENTE EN EXAMEN

Cuando la diferencia entre exponentes es mayor que la cantidad de bits de mantisa disponibles, al correr la mantisa del más chico a la derecha, todos sus bits significativos se caen fuera del ancho de la mantisa. El resultado queda idéntico al número más grande — no es un error de cálculo, es consecuencia del ancho finito de la mantisa.

Formato 16 bits (1+5+10, sesgo=15) — sumar $A=1.000011 \times 2^{-1}$ y $B=1.00101 \times 2^{12}$:

Diferencia de exponentes: $12 - (-1) = 13$ — mayor que los 10 bits de mantisa. Al correr A 13 lugares, TODOS sus bits caen fuera.

Resultado A+B = idéntico a B solo. A quedó completamente absorbido.

14. Error de redondeo

Cota de error para $1.f \times 2^e$ con 23 bits de mantisa (simple precisión): mitad del ULP = $2^{(e-24)}$.

Rango	Exponente e	Cota de error
[0,1]	e=0	$\leq 2^{-24}$
[2,4)	e=1	$\leq 2^{-23}$
[128,256)	e=7	$\leq 2^{-17}$
$[2^{20}, 2^{21})$	e=20	$\leq 2^{-4}$

Conclusión clave: el error **absoluto** crece con la magnitud (proporcional a 2^e), pero el error **relativo** se mantiene $\sim$ constante ($\sim 2^{-24}$). Esto explica el “número absorbido”: cuando dos números difieren mucho en magnitud, el error relativo del grande ya supera el valor absoluto completo del chico.

15. Ejercicios de repaso rápido (con respuesta)

Tapá la columna de respuesta y resolvé antes de mirar.

Enunciado	Respuesta
Representar -12 en VA+S de 8 bits	10001100
Representar -12 en C1 de 8 bits	11110011
Representar -12 en C2 de 8 bits	11110100
Decodificar 11101001 en C2 (8 bits)	invierto $\rightarrow$ 00010110, +1 $\rightarrow$ 00010111 = 23 $\rightarrow$ -23
Rango de C2 en 6 bits	-32 a +31
¿Cuántos ceros tiene C1?	dos (+0 y -0)
Peso del bit de signo en C2, 8 bits	-128
Peso del bit de signo en C1, 8 bits	-127
Desplazamiento(-12), 8 bits, d=128	$-12+128=116 = 01110100$
V=1 en suma C2: ¿condición mínima?	mismo signo en ambos operandos, signo distinto en el resultado
Sesgo del exponente en simple precisión	127
Sesgo del exponente en media precisión	15
Bits de mantisa en simple precisión	23 (+1 implícito = 24 de precisión real)
exp=0, mantisa=0 en IEEE754	cero (+0 o -0 según el signo)
exp=todo1, mantisa $\neq$ 0 en IEEE754	NaN

16. Resumen ultra-rápido (2 minutos)

- **VA+S:** signo aparte + magnitud tal cual. Dos ceros. Rango simétrico.
- **C1:** negativo = invertir todos los bits. Dos ceros. Signo pesa $-(2^{n-1}-1)$.
- **C2:** negativo = C1+1. Un solo cero. Signo pesa -2^{n-1} . Usado por el hardware.
- **Desplazamiento:** se guarda $N+2^{n-1}$ como sin signo. Truco: $\text{Desplazamiento}(N) = C2(N)$ con MSB invertido.
- **Flags:** Z=resultado 0. N=MSB del resultado. C=acarreo sin signo. V=overflow con signo.
- **Punto fijo:** multiplicar por 2^k , convertir a C2, listo.

- **IEEE 754:** signo + exponente (Desplazamiento) + mantisa (1. implícito).
exp=0→cero/desnormalizado; exp=todo1→infinito/NaN.
- **Trampa recurrente:** el “número absorbido” al sumar exponentes muy distintos.